Python Intermediate day 1 Recap

1) **Modules – The Big Picture**
2) Module Coding Basics
3) Packages & Relative Imports
4) Advanced Module Topics

3 Pretest question on OOP

1. Email *

2. Name

3. **Why are modules used in Python?** 1 point

Mark only one oval.

- A. To speed up CPU execution
- B. To organize code into reusable components
- C. To replace classes
- D. To avoid writing functions

4. **. What happens FIRST when a module is imported?** 1 point

Mark only one oval.

- A. The module is executed
- B. Bytecode is loaded
- C. The module is searched for in sys.path
- D. The module is cached

5. **Where is compiled bytecode usually stored?** 1 point

Mark only one oval.

- A. Same folder as Python executable
- B. site-packages
- C. __pycache__
- D. /tmp

6. **What is the main risk of using from module import *?** 1 point

Mark only one oval.

- A. Slower execution
- B. Namespace pollution
- C. Syntax errors
- D. Memory leaks

7. **When is module code executed?** 1 point

Mark only one oval.

- A. Every time a function is called
- B. Only when imported the first time
- C. Every time it's imported
- D. Only in interactive mode

8. **What file makes a directory a traditional Python package?** 1 point

Mark only one oval.

- A. main.py
- B. package.py
- C. __init__.py
- D. setup.cfg

9. **Which import is ABSOLUTE?** 1 point

Mark only one oval.

- A. from .utils import helper
- B. from ..core import base
- C. from myapp.utils import helper
- D. import .utils

10. **Namespace packages allow:** 1 point

Mark only one oval.

- A. Multiple directories for one package
- B. Faster imports
- C. Hidden variables
- D. Automatic reloading

11. **What does _variable conventionally indicate?** 1 point

Mark only one oval.

- A. Constant value
- B. Public API
- C. Internal or private use
- D. Deprecated code

12. **What is the purpose of if __name__ == "__main__":?** 1 point

Mark only one oval.

- A. Speed optimization
- B. Prevent imports
- C. Allow script + module behavior
- D. Define a package

13. **What is a class in Python?** 1 point

Mark only one oval.

- A. A function
- B. A template for creating objects
- C. A module
- D. A variable

14. **What does method overriding mean?** 1 point

Mark only one oval.

- A. Deleting parent methods
- B. Changing method parameters
- C. Redefining a parent method in child
- D. Calling parent methods

15. **What enables polymorphism in Python?** 1 point

Mark only one oval.

- A. Dynamic typing
- B. Function overloading only
- C. Compilation
- D. Bytecode caching